

GAME PROGRAMMING PATTERNS

ステートパターン

状態ごとの処理を整理し、見通しの良いコードを書く

対象：3年生

STAT
E

参考資料について

- この授業は、Unity 公式の eBook 「Level up your code with game programming patterns」を参考にしています
- サンプルプロジェクト（GitHub）にも、ステートパターンの実装例が収録されています
- 実践パートでは、実際に Unity プロジェクトを配布して手を動かします



参考 eBook： Unity3D.jp

<https://unity3d.jp/game/game-ebooks/level-up-your-code-with-game-programming-patterns/>



サンプルプロジェクト： GitHub

<https://github.com/Unity-Technologies/game-programming-patterns-demo/>

LESSON

ステートパターン

目標：状態管理を整理し、実際のプロジェクトで手を動かして理解する



このコマの目標

- if 文・ switch 文で状態を管理することの問題点を体感する
- ステートパターンの考え方（Enter ・ Update ・ Exit）を理解する
- リファクタリング前後のコードを見比べ、実際のプロジェクトで動きを確認する
- 状態の追加（コヨーテタイム・ヒップドロップ）とジャンプ周りの改善を自分の手で行う

悪い例： switch 文で状態を管理する

c# Program.cs

```
1 public enum CharacterState
2 {
3     Idle,
4     Walk,
5     Jump
6 }
7 public class NaiveCharacter : MonoBehaviour
8 {
9     private CharacterState state;
10    void Update()
11    {
12        switch (state)
13        {
14            case CharacterState.Idle:
15                UpdateIdle();
16                break;
17            case CharacterState.Walk:
18                UpdateWalk();
19                break;
20            case CharacterState.Jump:
21                UpdateJump();
22                break;
23        }
24    }
25 }
```

悪い例の問題点

- 状態が増えるほど、switch 文がどんどん長くなっていく
- 1 つの Update メソッドに、すべての状態の処理が詰め込まれてしまう
- 新しい状態（例：しゃがむ、攻撃する）を追加するたびに、この巨大な switch 文を修正する必要がある
- 「どこからどこへ状態が移れるか」というルールが、コードのあちこちに散らばりやすい

超悪い例：すべて if 文でチェックする

c# Program.cs

```
1 public class SuperNaiveMario : MonoBehaviour
2 {
3     private bool isIdle = true;
4     private bool isRunning;
5     private bool isJumping;
6
7     void Update()
8     {
9         if (isIdle && !isRunning && !isJumping) UpdateIdle();
10        if (isRunning && !isJumping) UpdateRun();
11        if (isJumping) UpdateJump();
12
13        if (!isJumping && Input.GetButtonDown("Jump"))
14        {
15            isJumping = true;
16            // isIdle と isRunning を false にし忘れると...?
17        }
18    }
19 }
```

状態遷移の図解



矢印は「状態遷移」を表す。同時にアクティブな状態は必ず 1 つだけ。

- 移動していなければ Idle、動けば Walk、ジャンプボタンを押せば Jump へ遷移する、といったイメージ。

ステートパターンとは

- 「状態」を1つずつ独立したクラスとして表現し、状態ごとの処理をそのクラスの中に閉じ込めるパターン
- 現在の状態オブジェクトを1つ持っておき、状態が変わったら参照先を差し替えるだけでよい
- 各状態は Enter（に入った瞬間）・Update（毎フレーム）・Exit（出る瞬間）の3つの区切りで処理を持つ
 - ー switch 文1つに全部書く代わりに、状態の数だけ小さなクラスに分ける、というイメージ

IState インターフェース

- すべての状態クラスが実装する共通のインターフェース。Enter ・ Update ・ Exit の 3 つを持つ。

C# Program.cs

```
1 public interface IState
2 {
3     void Enter();
4     void Update();
5     void Exit();
6 }
```

POINT

Enter = 状態に入った瞬間、Update = その状態にいる間ずっと、Exit = 状態から出る瞬間、に呼ばれる。

Unity プロジェクトの準備

- ① Unity 6 で新規 2D プロジェクトを、各自作成する
- ② public > 高等部 > 26_08_21 のファイルをすべて、Assets 直下に取り込む
- ③ InGameDemo シーンを開く
- ④ Player オブジェクトのレイヤーを Player に変更する

POINT

ここから先は、実際のプロジェクト内のスクリプトを見ながら進めます。

Player のプログラム①：フィールド

- 今、Player オブジェクトについているのはこちらのスクリプト。状態ごとに bool 変数を持つ、素朴な作りです。

```
C# UnRefactCharacter.  
CS  
1 public class UnRefactCharacter : MonoBehaviour  
2 {  
3     public float moveSpeed = 5f;  
4     public float jumpPower = 8f;  
5     [SerializeField] float rayRange = 0.8f;  
6     public bool isGrounded;  
7     public bool isIdle = true;  
8     public bool isWalking;  
9     public bool isJumping;  
10    public bool isHipDropping;  
11    public bool isFalling;  
12    public Rigidbody2D rb;  
13    public Animator anim;  
14    public SpriteRenderer sr;
```

Player のプログラム②: Awake ・ Start

C# UnRefactCharacter.
cs (続き)

```
1  void Awake()  
2  {  
3      rb = GetComponent<Rigidbody2D>();  
4      anim = GetComponent<Animator>();  
5      sr = GetComponent<SpriteRenderer>();  
6  }  
7  
8  void Start()  
9  {  
10     CameraFollow.Instance.SetTarget(transform);  
11 }
```

Player のプログラム③：接地判定とジャンプ中

C# UnRefactCharacter.
cs (続き)

```
1 void Update()
2 {
3     int layerMask = ~(1 << LayerMask.NameToLayer("Player"));
4     RaycastHit2D hit = Physics2D.Raycast(
5         transform.position, Vector2.down, rayRange, layerMask);
6     isGrounded = hit.collider != null;
7     float input = Input.GetAxisRaw("Horizontal");
8
9     if (isJumping)
10    {
11        if (rb.linearVelocity.y < 0)
12        {
13            isJumping = false;
14            isFalling = true;
15            anim.Play("fall-Animation");
16        }
17    }
```

Player のプログラム④：落下中と歩行の判定開始

C# UnRefactCharacter.
cs (続き)

```
1     else if (isFalling)
2     {
3         if (isGrounded)
4         {
5             isFalling = false;
6             isIdle = true;
7         }
8     }
9     else if (isWalking || isIdle)
10    {
11        rb.linearVelocity = new Vector2(
12            input * moveSpeed, rb.linearVelocity.y);
13
14        if (Mathf.Abs(input) > 0.1f)
15        {
```

Player のプログラム⑤：歩行の分岐と向き反転

C# UnRefactCharacter.
cs (続き)

```
1         isWalking = true; isIdle = false;
2         anim.Play("run-Animation");
3     }
4     else
5     {
6         isIdle = true; isWalking = false;
7         anim.Play("Idle-Animation");
8     }
9
10    if (input > 0) { sr.flipX = false; }
11    else if (input < 0) { sr.flipX = true; }
```

Player のプログラム⑥： ジャンプ入力と落下トリガー

C# UnRefactCharacter.
cs (続き)

```
1      if (Input.GetButtonDown("Jump") && isGrounded)
2      {
3          isJumping = true; isWalking = false; isIdle = false;
4          rb.linearVelocity = new Vector2(rb.linearVelocity.x, jumpPower);
5          anim.Play("jump-Animation");
6      }
7
8      if (isGrounded == false)
9      {
10         isFalling = true;
11         anim.Play("fall-Animation");
12     }
13 }
14 }
```

POINT

isHipDropping は宣言されているだけで、まだ Update の中で使われていない点にも注目。

State に直したものの解説

- bool 変数と 1 つの巨大な Update を、状態ごとのクラスに分割したものがこちらです。
- IdleState ・ WalkState ・ JumpState ・ FallState の 4 つに分かれ、それぞれが Enter ・ Update ・ Exit を持つ
- ジャンプの力はここではまだ一定（jumpPower 固定）。この後の実践で、少しずつ改良していきます

POINT

処理の中身（やっていること）は UnRefactCharacter と同じ。整理の仕方だけが違います。

MarioCharacter クラス①: フィールド

c# Program.cs

```
1 public class MarioCharacter : MonoBehaviour
2 {
3     public StateMachine stateMachine { get; private set; }
4     public IdleState idleState;
5     public WalkState walkState;
6     public JumpState jumpState;
7     public FallState fallState;
8
9     public float moveSpeed = 5f;
10    public float jumpPower = 8f;
11
12    [SerializeField] float rayRange = 0.8f;
13    public bool isGrounded;
14
15    public Rigidbody2D rb;
16    public Animator anim;
17    public SpriteRenderer sr;
```

MarioCharacter クラス②: Awake

c# Program.cs

```
1 void Awake()  
2 {  
3     rb = GetComponent<Rigidbody2D>();  
4     anim = GetComponent<Animator>();  
5     sr = GetComponent<SpriteRenderer>();  
6  
7     stateMachine = new StateMachine();  
8     idleState = new IdleState(this);  
9     walkState = new WalkState(this);  
10    jumpState = new JumpState(this);  
11    fallState = new FallState(this);  
12 }
```

MarioCharacter クラス③: Start ・ Update

c# Program.cs

```
1 void Start()
2 {
3     // 最初は Idle 状態からスタート
4     stateMachine.Initialize(idleState);
5
6     // カメラ追従設定
7     CameraFollow.Instance.SetTarget(transform);
8 }
9
10 void Update()
11 {
12     // 着地判定の更新
13     CheckGround();
14
15     // 現在の状態の Update を呼ぶだけでよい
16     stateMachine.Update();
17 }
```

MarioCharacter クラス④：接地判定

c# Program.cs

```
1 private void CheckGround()
2 {
3     int layerMask = ~(1 << LayerMask.NameToLayer("Player"));
4     RaycastHit2D hit = Physics2D.Raycast(
5         transform.position, Vector2.down, rayRange, layerMask);
6
7     if (hit.collider != null)
8     {
9         isGrounded = true;
10    }
11    else
12    {
13        isGrounded = false;
14    }
15 }
16
17 private void OnDrawGizmosSelected()
18 {
19     Debug.DrawRay(transform.position, Vector2.down * rayRange, Color.red);
20 }
```

IdleState の実装

c# IdleState.cs

```
1 public class IdleState : IState
2 {
3     private MarioCharacter mario;
4     public IdleState(MarioCharacter mario)
5     {
6         this.mario = mario;
7     }
8     public void Enter()
9     {
10         mario.anim.Play("Idle-Animation");
11     }
12     public void Update()
13     {
14         float input = Input.GetAxisRaw("Horizontal");
15         if (Mathf.Abs(input) > 0.1f)
16         {
17             mario.stateMachine.TransitionTo(mario.walkState);
18         }
19         else if (Input.GetButtonDown("Jump") && mario.isGrounded)
20         {
21             mario.stateMachine.TransitionTo(mario.jumpState);
22         }
23     }
24     public void Exit() { }
25 }
```

WalkState の実装①：移動と状態遷移

c# WalkState.cs

```
1 public void Enter()
2 {
3     mario.anim.Play("run-Animation");
4 }
5 public void Update()
6 {
7     float input = Input.GetAxisRaw("Horizontal");
8     mario.rb.linearVelocity = new Vector2(
9         input * mario.moveSpeed, mario.rb.linearVelocity.y);
10
11     if (Mathf.Abs(input) < 0.1f)
12     {
13         mario.stateMachine.TransitionTo(mario.idleState);
14     }
15     else if (Input.GetButtonDown("Jump") && mario.isGrounded)
16     {
17         mario.stateMachine.TransitionTo(mario.jumpState);
18     }
```

WalkState の実装②：向き反転と落下トリガー

- 地面から離れた瞬間に、FallState へ遷移する。

C# WalkState.cs (続き)

```
1  if (input > 0)
2  {
3      mario.sr.flipX = false;
4  }
5  else if (input < 0)
6  {
7      mario.sr.flipX = true;
8  }
9
10 if (mario.isGrounded == false)
11 {
12     mario.stateMachine.TransitionTo(mario.fallState);
13 }
```

JumpState の実装

c# JumpState.cs

```
1 public class JumpState : IState
2 {
3     private MarioCharacter mario;
4     public JumpState(MarioCharacter mario)
5     {
6         this.mario = mario;
7     }
8     public void Enter()
9     {
10         mario.rb.linearVelocity = new Vector2(
11             mario.rb.linearVelocity.x, mario.jumpPower);
12         mario.anim.Play("jump-Animation");
13     }
14     public void Update()
15     {
16         if (mario.rb.linearVelocity.y <= 0)
17         {
18             mario.stateMachine.TransitionTo(mario.fallState);
19         }
20     }
21     public void Exit() { }
22 }
```

FallState の実装

c# FallState.cs

```
1 public class FallState : IState
2 {
3     private MarioCharacter mario;
4     public FallState(MarioCharacter mario)
5     {
6         this.mario = mario;
7     }
8     public void Enter()
9     {
10         mario.anim.Play("fall-Animation");
11     }
12     public void Update()
13     {
14         if (mario.isGrounded)
15         {
16             mario.stateMachine.TransitionTo(mario.idleState);
17         }
18     }
19     public void Exit() { }
20 }
```

実践①： ジャンプ力を調整する

- ジャンプボタンを短く押すと低いジャンプ、長く押し続けると高いジャンプになるようにします。
- ボタンを押していただける最大時間（jumpButtonTime）を決めておき、実際に押していた時間の割合で力を調整する
- 力を与え終わったら、あとは通常どおり下降を待って FallState へ遷移する

POINT

ここから先は、実際にスクリプトを書き換えながら一緒に進めます。

ジャンプ力①: フィールドの追加と Enter

C# MarioCharacter.cs
(追記)

```
1 public float jumpButtonTime = 0.15f;  
2 public float jumpButtonCounter;
```

C# JumpState.cs (Enter
書き換え)

```
1 public void Enter()  
2 {  
3     mario.anim.Play("jump-Animation");  
4     mario.jumpButtonCounter = mario.jumpButtonTime;  
5 }
```

POINT

Enter では、もう速度を与えない。カウンターをセットするだけに変わる点に注目。

ジャンプ力②: Update の書き換え

- カウンターが0になるか、ボタンが離されるまでの経過時間に応じて、上向きの力を少しずつ大きくしていく。

C# JumpState.cs (Update の一部)

```
1  if (mario.jumpButtonCounter != 0)
2  {
3      mario.jumpButtonCounter -= Time.deltaTime;
4      if (mario.jumpButtonCounter < 0 || Input.GetButtonUp("Jump"))
5      {
6          float power = mario.jumpPower *
7              (mario.jumpButtonTime - mario.jumpButtonCounter) / mario.jumpButtonTime;
8          mario.rb.linearVelocity = new Vector2(mario.rb.linearVelocity.x, power);
9          mario.jumpButtonCounter = 0;
10     }
11 }
```

ジャンプ力③： 下降への切り替え

- 力を与え終えた（jumpButtonCounter が 0 の）状態で、速度が下向きになったら FallState へ遷移する。

```
JumpState.cs (U  
C# pdate の続き)  
  
1 else  
2 {  
3     if (mario.rb.linearVelocity.y <= 0)  
4     {  
5         mario.stateMachine.TransitionTo(mario.fallState);  
6     }  
7 }
```

実践②：ヒップドロップステートを追加する

- 空中で下入力をする、勢いよく地面へ降下する「ヒップドロップ」を新しい状態として追加します。
- UnRefactCharacter の isHipDropping フィールドは、実は使われなまま残っていたことを思い出してください
- 新しい状態を 1 つ増やすだけで対応できる、というのがステートパターンの強みです

ヒップドロップ①: HipDropState の追加

C# MarioCharacter.cs
(追記)

```
1 public float hipDropSpeed = 12f;
```

C# HipDropState.cs

```
1 public class HipDropState : IState
2 {
3     private MarioCharacter mario;
4     public HipDropState(MarioCharacter mario)
5     {
6         this.mario = mario;
7     }
8     public void Enter()
9     {
10         mario.rb.linearVelocity = new Vector2(0f, -mario.hipDropSpeed);
11         mario.anim.Play("hipdrop-Animation");
12     }
13     public void Update()
14     {
15         if (mario.isGrounded)
16         {
17             mario.stateMachine.TransitionTo(mario.idleState);
18         }
19     }
20     public void Exit() { }
21 }
```

ヒップドロップ②：発動条件の追加

- 空中（Jump・Fall中）に下入力をする、ヒップドロップへ遷移する。

```
JumpState.cs /  
c# FallState.cs (追記)  
1  if (Input.GetAxisRaw("Vertical") < -0.5f)  
2  {  
3      mario.stateMachine.TransitionTo(mario.hipDropState);  
4  }
```

POINT 地面に着いたら HipDropState の Update で Idle へ戻る。着地時のエフェクトを足すと迫力が出ます。

実践③：コヨーテタイムの追加

- 足場の端から落ち始めた直後の一瞬だけ、まだジャンプできるようにする「猶予時間」を追加します。
- 地面にいる間はタイマーをリセットし続け、離れた瞬間からタイマーが減っていくようにする
- タイマーが残っている間だけ、ジャンプ入力を受け付ける

コヨーテタイム①：フィールドの追加

C# MarioCharacter.cs
(追記)

```
1 public float coyoteTime = 0.15f;  
2 public float coyoteTimeCounter;
```

コヨーテタイム②: FallState の Update 書き換え

C# FallState.cs (Update 書き換え)

```
1 public void Update()
2 {
3     mario.coyoteTimeCounter -= Time.deltaTime;
4     if (mario.coyoteTimeCounter > 0 && Input.GetButtonDown("Jump"))
5     {
6         mario.stateMachine.TransitionTo(mario.jumpState);
7     }
8     if (mario.isGrounded)
9     {
10        mario.stateMachine.TransitionTo(mario.idleState);
11    }
12 }
```

コヨーテタイム③: WalkState でのリセット

- 地面から離れて FallState へ遷移する瞬間に、タイマーを最大値まで戻しておく。

C# WalkState.cs (続き・書き換え)

```
1  if (mario.isGrounded == false)
2  {
3      mario.stateMachine.TransitionTo(mario.fallState);
4      mario.coyoteTimeCounter = mario.coyoteTime;
5  }
```

完成した状態遷移の図解

Walk

Idle

Jump

HipDrop

Fall

- Idle/Walk ⇄ Jump ⇄ Fall ⇄ HipDrop という形で、状態が少しずつつながっていく。
- コヨーテタイムは、既存の状態の中の「処理」として追加されている点に注目。

問題： ジャンプ中の空中操作を追加する

- 今の JumpState ・ FallState では、ジャンプ中の左右の速度がジャンプした瞬間のまま固定されています。
- マリオのように、空中でも左右キーである程度速度を調整できるようにしてみましょう。
- ヒント： JumpState ・ FallState の Update で、 WalkState と同じように入力を読み取り、 moveSpeed より少し弱い力で `rb.linearVelocity.x` を変化させてみる。

注意

地上と全く同じ速さで操作できてしまうと、ジャンプの意味が薄れる点に注意。

長所と短所

長所

- 状態ごとの処理が独立したクラスに分かれ、見通しが良くなる
- 新しい状態を追加しても、既存の状態にほとんど影響しない
- 「どの状態からどの状態へ移れるか」を状態クラスの中に閉じ込められる

短所

- 状態の数だけクラスが増え、ファイル数が多くなる
- 状態が少ない・変化がほぼ無い場合は、if文の方がシンプルなこともある
- 状態同士のやり取りが複雑になると、遷移のルールを追うのが大変になる

このコマのまとめ

- **ステートパターン=状態ごとの処理を、独立したクラスに分けて管理するパターン**
- IState（Enter・Update・Exit）を実装し、StateMachineで現在の状態を管理する
- リファクタリング前後を見比べることで、「整理の仕方」が変わるだけで中身は同じだと確認できた
- ジャンプ力の調整・ヒップドロップの追加・コヨーテタイムの追加という3つの実践を通して、自分の手で機能を拡張した

お疲れさまでした

今日解いた問題の続きとして、しゃがむ・攻撃するなどの状態を追加してみると、パターンの効果をさらに実感できます。